

PROGRAMAÇÃO WEB

AULAS TEÓRICAS

ANO LECTIVO DE 2024/25

1° SEMESTRE

PROGRAMA PREVISTO



1. **Conceitos de programação em *C# 12***
   - 1.1. Conceitos Básicos
   - 1.2. Conceitos Avançados
2. **Construção de Websites em .NET Core 8**
   - 2.1. Introdução ao *.Net Core 8*
   - 2.2. Estrutura de uma aplicação *.NET Core 8*
   - 2.3. *Blazor* e Modelos de Hospedagem *Blazor* no *.Net Core 8*
     - 2.3.1 *.Net Core 8 Blazor Server*
     - 2.3.2 *.Net Core 8 Blazor WebAssembly*
     - *2.3.3 .Net Core 8 Blazor Hybrid*

PROGRAMA PREVISTO - CONTINUAÇÃO

2.4. .Net 8 MAUI

2.5. .Net 8 Razor Class Library

2.6. .Net 8 Fluent UI

2.7. Arquitecturas para desenvolvimento Web

2.8. API, Minimal API e Microserviços

2.9. Docker no .Net Core 8

2.10. SignalR

2.11. Azure Service Bus

2.12. Cloud Azure

PROGRAMA PREVISTO - CONTINUAÇÃO

3. Manipulação de Dados em .Net Core 8

- 3.1. .Net Core 8 Entity Framework
- 3.2. Migrations
- 3.3. Utilização de LINQ

4. Segurança em .Net Core 8

- 4.1. Segurança em Websites
- 4.2.Validação de utilizadores
- 4.3. .NET Core 8 Identity Framework

PROGRAMA PREVISTO



1.Conceitos de programação em C# 12

1.1. Conceitos Básicos

1. CONCEITOS DE PROGRAMAÇÃO EM C# *12*



- História do C#
- Desenvolvida pela Microsoft
- Baseada no C (Mistura de C++ e Java)
  - Orientada a Objetos
- Principal linguagem de programação para framework .NET
- Criador: Anders Hejlsberg
- Introduzido C#1.0 com VS2002, atualmente vai na versão C# 12.0 com VS2022 (ASP.Net Core 8)
- Possui um elevado nível de abstração

1. CONCEITOS DE PROGRAMAÇÃO EM C# 12



- O C# foi introduzido pela primeira vez no início dos anos 2000 pela Microsoft como parte da sua iniciativa .NET.
- A linguagem foi criada por uma equipa liderada por Anders Hejlsberg, um proeminente engenheiro de software conhecido pelo seu trabalho no Turbo Pascal e Delphi.

1. CONCEITOS DE PROGRAMAÇÃO EM *C# 12*



- O objetivo era desenvolver uma linguagem moderna e orientada a objetos que pudesse competir com o Java e fornecer uma estrutura robusta para a construção de aplicações Windows.
- C# (pronuncia-se “C-sharp”) é uma linguagem de programação versátil e poderosa desenvolvida pela Microsoft como parte da sua iniciativa .NET. Concebido para ser simples, moderno e orientado a

1. CONCEITOS DE PROGRAMAÇÃO EM *C# 12*



- O objetivo era desenvolver uma linguagem moderna e orientada a objetos que pudesse competir com o Java e fornecer uma estrutura robusta para a construção de aplicações Windows.
- C# é uma linguagem de programação versátil e poderosa desenvolvida pela Microsoft como parte da sua iniciativa .NET.

1. CONCEITOS DE PROGRAMAÇÃO EM C# 12



- Concebido para ser simples, moderno e orientado a objetos, o C# é utilizado para construir uma grande variedade de aplicações, desde software de desktop a serviços Web e aplicações móveis.

- A sua estrutura robusta, o amplo suporte de bibliotecas e a integração perfeita com o ecossistema . NET fazem do C# uma escolha popular entre os programadores para a criação de aplicações escaláveis e de alto desempenho.

1. CONCEITOS DE PROGRAMAÇÃO EM C# 12



- **Orientado a objetos:** o C# segue o paradigma da programação orientada a objetos (OOP), que promove a reutilização, a modularidade e a flexibilidade do código através de conceitos como classes, herança, polimorfismo e encapsulamento.

- **Type-Safe:** O C# é uma linguagem strong typing, o que ajuda a detetar erros em tempo de compilação, reduzindo os erros em tempo de execução e melhorando a fiabilidade do código.

1. CONCEITOS DE PROGRAMAÇÃO EM C# 12



- **Orientado a Componentes:** O C# suporta o desenvolvimento de componentes de software, facilitando a criação de código reutilizável e de fácil manutenção.

- **Interoperabilidade:** o C# oferece uma excelente interoperabilidade com outras linguagens e plataformas, facilitando a integração com código e bibliotecas existentes.

1. CONCEITOS DE PROGRAMAÇÃO EM C# *12*



- **Programação Assíncrona:** o C# inclui as palavras-chave *async* e *await*, permitindo aos programadores escrever código assíncrono mais facilmente, melhorando o desempenho e a capacidade de resposta da aplicação.

- **Rich Library Support:** o .NET Framework e o .NET Core oferecem uma vasta gama de bibliotecas e API, simplificando tarefas como E/S de ficheiros, rede, acesso a bases de dados e muito mais.



Porquê usar C#?

1. Versátil e poderoso

C# é uma linguagem versátil que pode ser utilizada para uma grande variedade de aplicações, incluindo aplicações web, móveis, desktop, jogos e aplicações baseadas na nuvem.
A sua versatilidade torna-o uma escolha ideal para diversos tipos de projetos.



2. **Programação Orientada a Objectos**

O C# é uma linguagem totalmente orientada a objetos, o que significa que incentiva práticas como a reutilização de código, a modularidade e a abstração.
Isto leva a um código mais limpo, mais sustentável e escalável.



3. **Rico Ecossistema e Bibliotecas**

O C# beneficia do extenso ecossistema .NET, que inclui uma vasta gama de bibliotecas e frameworks que simplificam muitas tarefas de programação comuns. Isto pode poupar tempo e esforço de desenvolvimento significativos.



4. **Desenvolvimento multiplataforma**

Com o . NET Core e agora o . NET 5/6/7/8, o C# permite o desenvolvimento entre plataformas, o que significa que se pode criar aplicações que funcionam em Windows, macOS e Linux. Isto expande a sua base de utilizadores potenciais e opções de implantação.



5. **Comunidade e apoio fortes**

O C# tem uma comunidade grande e ativa, com diversos recursos disponíveis para aprendizagem e resolução de problemas.

Quer se esteja à procura de tutoriais, fóruns ou projetos de código aberto, a comunidade C# oferece um amplo suporte.



6. Recursos de linguagem moderna

O C# evolui consistentemente com os recursos de programação modernos. Isto inclui programação assíncrona com async/await, consulta integrada na linguagem (LINQ), correspondência de padrões e muito mais, que ajuda os programadores a escrever código mais eficiente e expressivo.



7. Integração com produtos Microsoft

O C# integra-se perfeitamente com outros produtos e serviços da Microsoft, como o Azure para computação em nuvem, o SQL Server para bases de dados e o Visual Studio para um ambiente de desenvolvimento integrado (IDE).

Isto pode agilizar o processo de desenvolvimento se já estiver no ecossistema da Microsoft.



8. **Desempenho**

O C# e o Run Time .NET são concebidos para um elevado desempenho.

Com os avanços no . NET Core e . NET 5/6/7/8, as aplicações escritas em C# podem ser executadas com sobrecarga mínima e alta eficiência, tornando-as adequadas para aplicações de desempenho crítico.



9. **Segurança**

O C# inclui uma variedade de funcionalidades concebidas para ajudar os programadores a escrever código seguro.
Isto inclui uma forte verificação de tipos, Garbage Collector e tratamento robusto de exceções.
Além disso, a estrutura . NET fornece várias ferramentas e bibliotecas para gerir questões de segurança, como autenticação e encriptação.



10. **Oportunidades de carreira**

O C# é amplamente utilizado na indústria, especialmente em ambientes empresariais.

Isto significa que aprender e dominar C# pode abrir inúmeras oportunidades de carreira, seja em desenvolvimento de software, desenvolvimento de jogos (usando Unity) ou até mesmo desenvolvimento Web (usando ASP.NET).



11. Desenvolvimento de jogos

O C# é a linguagem principal utilizada com o Unity, um dos motores de desenvolvimento de jogos mais populares.

Isto torna o C# uma linguagem crucial para os aspirantes a programadores de jogos, fornecendo ferramentas e bibliotecas personalizadas para a criação de jogos interativos e envolventes.



12. Escalabilidade e Manutenção

As características do C# suportam a escrita de código limpo, sustentável e escalável. Isto é particularmente importante para grandes projetos e aplicações de nível empresarial, onde a manutenção e a escalabilidade a longo prazo são considerações críticas.



Futuro do C#

1. Evolução e inovação contínuas

O C# é conhecido pelas suas atualizações regulares e pela introdução de funcionalidades modernas. A Microsoft demonstrou um forte compromisso com a evolução da linguagem, tornando-a mais poderosa, expressiva e amigável para os programadores a cada lançamento.

As futuras versões do C# devem continuar esta tendência, incorporando mais características que simplificam a codificação e melhoram o desempenho.



2. Capacidades multiplataforma melhoradas

Com o ecossistema .NET, especialmente o .NET 8 e posteriores, o C# deverá tornar-se ainda mais robusto no desenvolvimento multiplataforma.
Isto significa um melhor suporte para a criação de aplicações que correm perfeitamente em Windows, macOS, Linux, iOS e Android, consolidando ainda mais o papel do C# em diversos ambientes de desenvolvimento.



3. Foco na cloud e desenvolvimento web

À medida que a computação em nuvem continua a crescer, o C# está posicionado para ser uma linguagem dominante para aplicações baseadas na nuvem, especialmente com a sua forte integração com o Microsoft Azure. Os melhoramentos no ASP.NET Core e no Blazor estão a facilitar a criação de aplicações Web, APIs e micro-serviços de alto desempenho, garantindo que o C# continua a ser importante no desenvolvimento Web.



4. Integração com IA e aprendizagem automática

O foco da Microsoft na integração da IA e da aprendizagem automática no seu ecossistema sugere que o C# terá um suporte crescente para estas tecnologias.

Com ferramentas como o ML.NET, os programadores já podem criar modelos de aprendizagem automática em C#.

As melhorias futuras provavelmente tornarão ainda mais fácil aproveitar as capacidades de IA e ML em aplicações C#.



5. IoT e Edge Computing

A Internet das Coisas (IoT) e a Edge Computing são campos em rápida expansão. Espera-se que o C# e o .NET desempenhem um papel significativo nestas áreas, oferecendo frameworks e bibliotecas que facilitam o desenvolvimento de soluções IoT. Isto será crucial à medida que mais dispositivos se ligarem e exigirem software eficiente, seguro e escalável.



6. Jogos e desenvolvimento de XR

O C# é a linguagem principal do Unity, um dos principais motores de desenvolvimento de jogos. Com o surgimento da realidade virtual (RV) e da realidade aumentada (RA), o C# continuará a ser uma linguagem crítica para a criação de experiências imersivas.

Melhorias contínuas no Unity e suporte para novo hardware garantirão a relevância do C# no desenvolvimento de jogos e XR (realidade alargada).



7. Melhorias de desempenho

A Microsoft trabalha continuamente para melhorar o desempenho do tempo de execução do .NET. As futuras atualizações da linguagem e do run time irão concentrar-se na otimização do desempenho, na redução do consumo de memória e na melhoria da eficiência global da aplicação.
Isto tornará o C# uma escolha ainda melhor para aplicações de alto desempenho.



8. Ferramentas melhoradas para desenvolvedores

O futuro do C# será também moldado pelos avanços nas ferramentas de programador. O Visual Studio, o Visual Studio Code e outros ambientes de desenvolvimento integrado (IDEs) continuarão a evoluir, oferecendo capacidades mais avançadas de depuração, teste e análise de código.
Estas melhorias tornarão mais fácil para os programadores escrever, testar e manter código C#.



9. Comunidade e ecossistema fortes

A comunidade C# continuará a contribuir para o crescimento da linguagem. As contribuições de código aberto, bibliotecas dirigidas pela comunidade e fóruns ativos desempenharão um papel vital na definição do futuro do C#.
Este esforço coletivo garante que o C# se mantém relevante e se adapta às novas necessidades dos programadores.

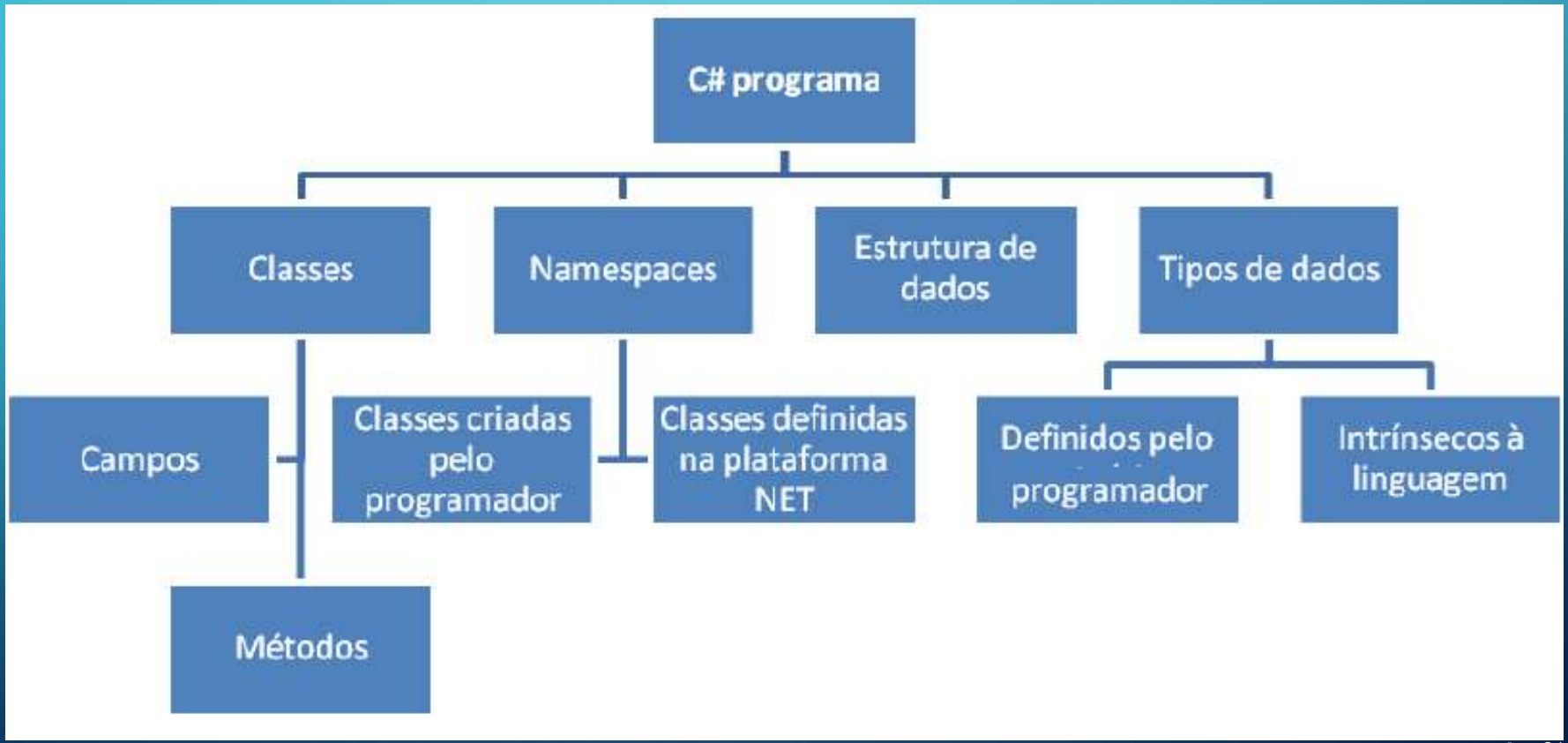


10. Oportunidades educativas e de carreira

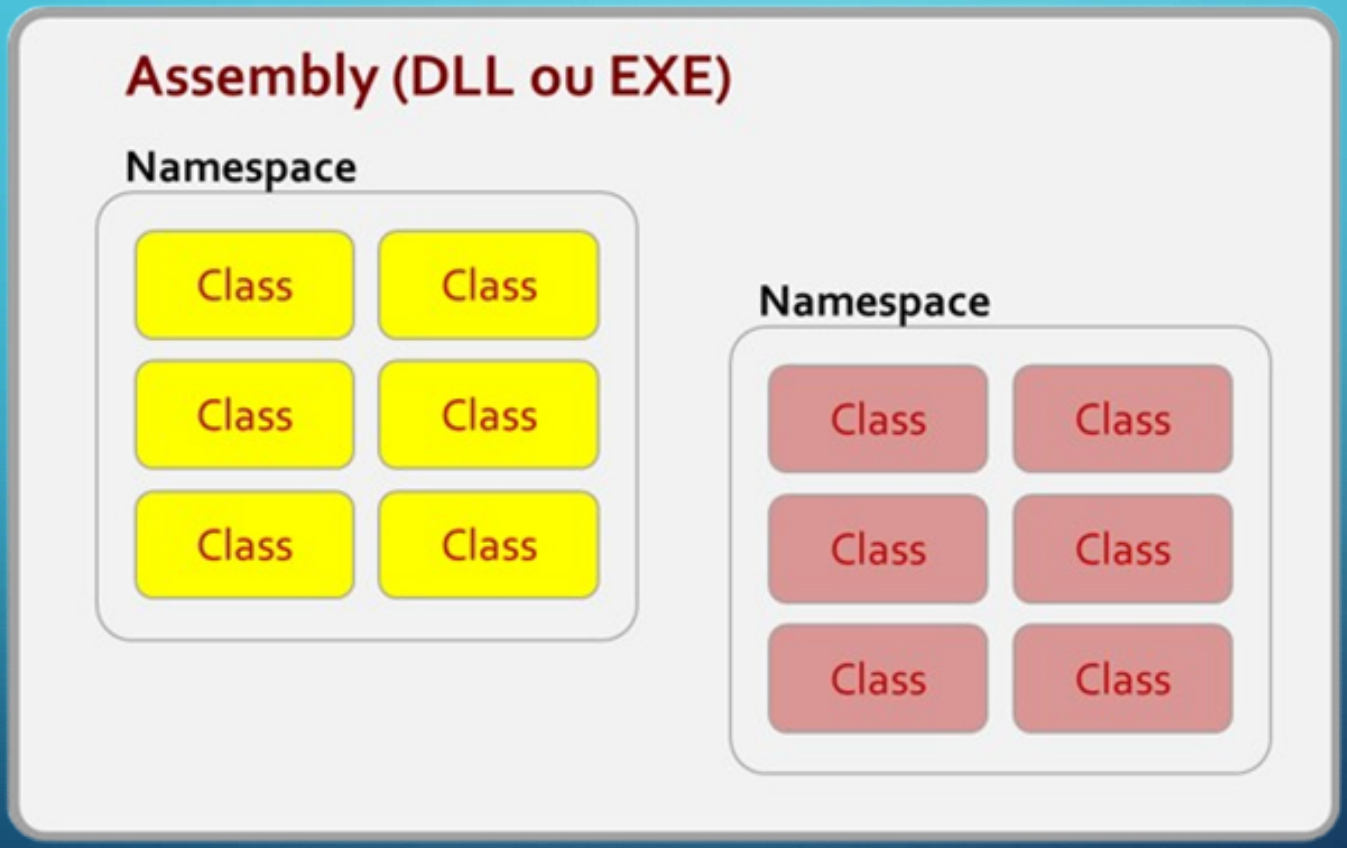
A proeminência do C# em vários setores garante que continuará a ser uma competência valiosa para os programadores.
As instituições de ensino e as plataformas online continuarão a oferecer cursos e certificações em C#, proporcionando amplas oportunidades para que os novos e experientes programadores possam melhorar as suas competências e progredir nas suas carreiras.

1. CONCEITOS DE PROGRAMAÇÃO EM C# *12*

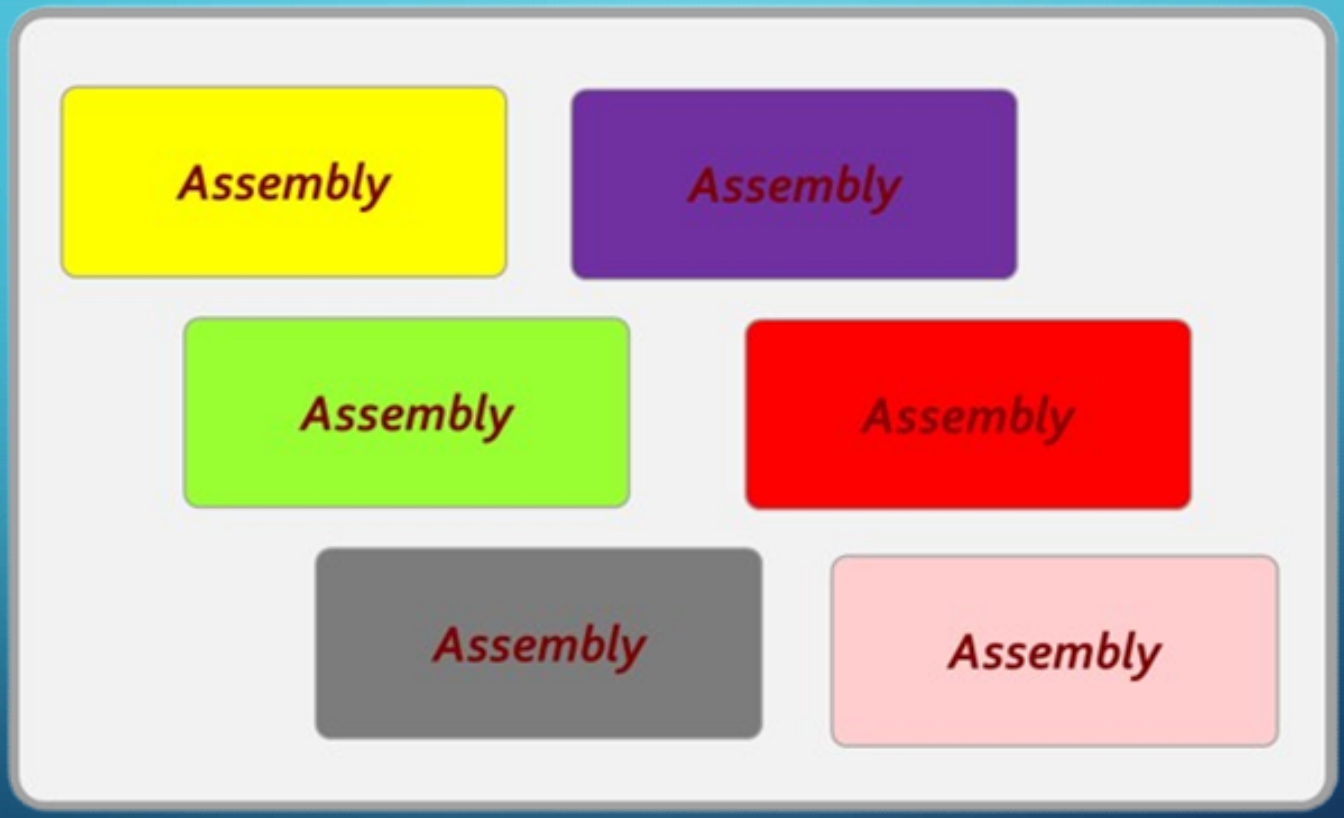
1.1. Conceitos Básicos



1. CONCEITOS DE PROGRAMAÇÃO EM C# *12*



1. CONCEITOS DE PROGRAMAÇÃO EM C# *12*





•Compreendendo os Assemblies

- Um assembly é uma biblioteca de código compilado utilizada por aplicações .NET para implementação, controlo de versões e segurança. É o bloco de construção das aplicações .NET e pode ser um ficheiro .exe (executável) ou . dll (biblioteca de ligação dinâmica).
  - Os assemblies contêm metadados sobre os tipos, recursos e outras informações necessárias para a execução da aplicação.



•Tipos de Assemblies

- Private Assembly: Utilizado por uma única aplicação e armazenado no diretório da aplicação.

- Shared Assembly: destinado a ser utilizado por várias aplicações e normalmente armazenado no Global Assembly Cache (GAC).



- **Strong Namming**
  - Um strong name confere ao assembly uma identidade exclusiva.

    Consiste no nome do assembly, número da versão, informação da cultura, uma chave pública e uma assinatura digital.

    A nomenclatura forte ajuda a garantir a integridade e a exclusividade de um assembly



- GAC (Global Assembly Cache)
  - O GAC é uma cache de código acessível por toda a máquina que armazena assemblies especificamente designados para serem partilhados por várias aplicações no computador.

  Fornece um local central para os assemblies partilhados, permitindo o controlo de versões e a execução lado a lado de diferentes versões de um assembly



- CTS - Common Type System
  - O CTS define como os tipos são declarados, utilizados e geridos no Common Language Runtime (CLR).
  - Garante que os objetos escritos em diferentes linguagens .NET possam interagir entre si.
  - O CTS suporta uma variedade de tipos e operações que se encontram na maioria das linguagens de programação.



- CLI - Common Language Infrastructure
  - CLI é uma especificação aberta desenvolvida pela Microsoft que descreve código executável e um ambiente de tempo de execução.
  - O seu objetivo é permitir que múltiplas linguagens de alto nível sejam utilizadas em diferentes plataformas de computador sem serem reescritas para arquiteturas específicas.
    - A estrutura . NET é uma implementação da CLI



- CLR – Common Language Runtime
  - O CLR é o motor de execução para aplicações .NET.
    - Fornece serviços como gestão de memória, gestão de threads, tratamento de exceções, recolha de "lixo", segurança e muito mais.
    - O CLR permite a interoperabilidade das linguagens e garante que a execução do código é gerida e otimizada.



- GC – Garbage Collector
- O *Garbage Collector* é uma funcionalidade de gestão automática de memória do CLR.
  - Ajuda a recuperar a memória ocupada por objetos que já não estão a ser utilizados pela aplicação, evitando o desperdicio de memória e optimizando o desempenho.



- GC.Collect()
  - GC.Collect() é um método que obriga o Garbage Collector a executar e tentar recuperar memória. Embora isto possa ser útil para a limpeza imediata, é geralmente melhor permitir que o Garbage Collector seja executado automaticamente para evitar problemas de desempenho.



- BCL – Base Class Library
  - A BCL é um conjunto básico de classes, interfaces e tipos por valor incluídos no Framework .NET.
    - Fornece funcionalidades e tipos fundamentais que são necessários para a maioria das aplicações, como coleções, E/S, threading e tipos de dados..



- FCL – Framework Class Library
- A FCL é uma coleção abrangente de classes, interfaces e tipos de valores reutilizáveis que são uma extensão da BCL.
  - Inclui tudo existente na BCL, além de bibliotecas adicionais para serviços Web, interação com base de dados e muito mais.



- **MSIL - Microsoft Intermediate Language**
  - MSIL é um conjunto de instruções independente da CPU que pode ser convertido em código nativo das diferentes CPU
  - Quando uma aplicação .NET é compilada, é traduzida para MSIL.
    - Esta linguagem intermédia permite que a aplicação seja executada em qualquer plataforma que possua uma implementação CLR compatível.



- CIL - Common Intermediate Language
  - CIL é outro termo para MSIL, utilizado para se referir à linguagem intermédia definida pela especificação CLI.
  - Serve o mesmo propósito que o MSIL no contexto do .NET.



- PE – Portable Executable
  - O PE é um formato de ficheiro para executáveis, código de objeto e DLLs em sistemas operativos Windows.
    - É utilizado para ficheiros EXE e DLL do Windows.
    - No contexto do . NET, o formato de ficheiro PE inclui um cabeçalho CLR que indica a presença de código gerido.



- JIT - Just-In-Time Compilation
  - A compilação JIT é o processo de conversão do código MSIL/CIL em código de máquina nativo antes da execução.
  - O compilador JIT do CLR compila o código da linguagem intermédia em código nativo dinamicamente, o que permite uma execução otimizada com base no ambiente atual.



- JIT - Just-In-Time Compilation
  - A compilação JIT é o processo de conversão do código MSIL/CIL em código de máquina nativo antes da execução.
  - O compilador JIT do CLR compila o código da linguagem intermédia em código nativo dinamicamente, o que permite uma execução otimizada com base no ambiente atual.

CARACTERÍSTICAS GERAIS

- Todos os ficheiros C# tem extensão `.cs`
- C# é *case sensitive*

```
Console.WriteLine("Programação em C#");

Console.Writeline("Programação em C#");
console.WriteLine("Programação em C#");
```

- O compilador C# ignora espaços adicionais
  - Sequência de espaços, carateres das tabs ,*cariage return*,…

CARACTERÍSTICAS GERAIS



- Cada *statement* C# deve terminar com um ponto e vírgula (;)
  - Podem existir vários *statements* numa só linha
    - Por questões de legibilidade, muda-se de linha no final de cada *statement*

- Um bloco de código C# está delimitado por { e }
  - Pode conter qualquer número de linhas de código ou nenhum
  - Estes não necessitam de ser acompanhados pelo ';'

CARACTERÍSTICAS GERAIS

- Comentários
  - Comentar uma só linha: // …

```
int x; // Isto é um comentário
```

  - Comentar mais do que uma linha: /* ….. ….. */

```
/* Isto..
      .....
       ...Também é um comentário! */
```

TIPO DE DADOS EM C#



Tipos de Dados em C#

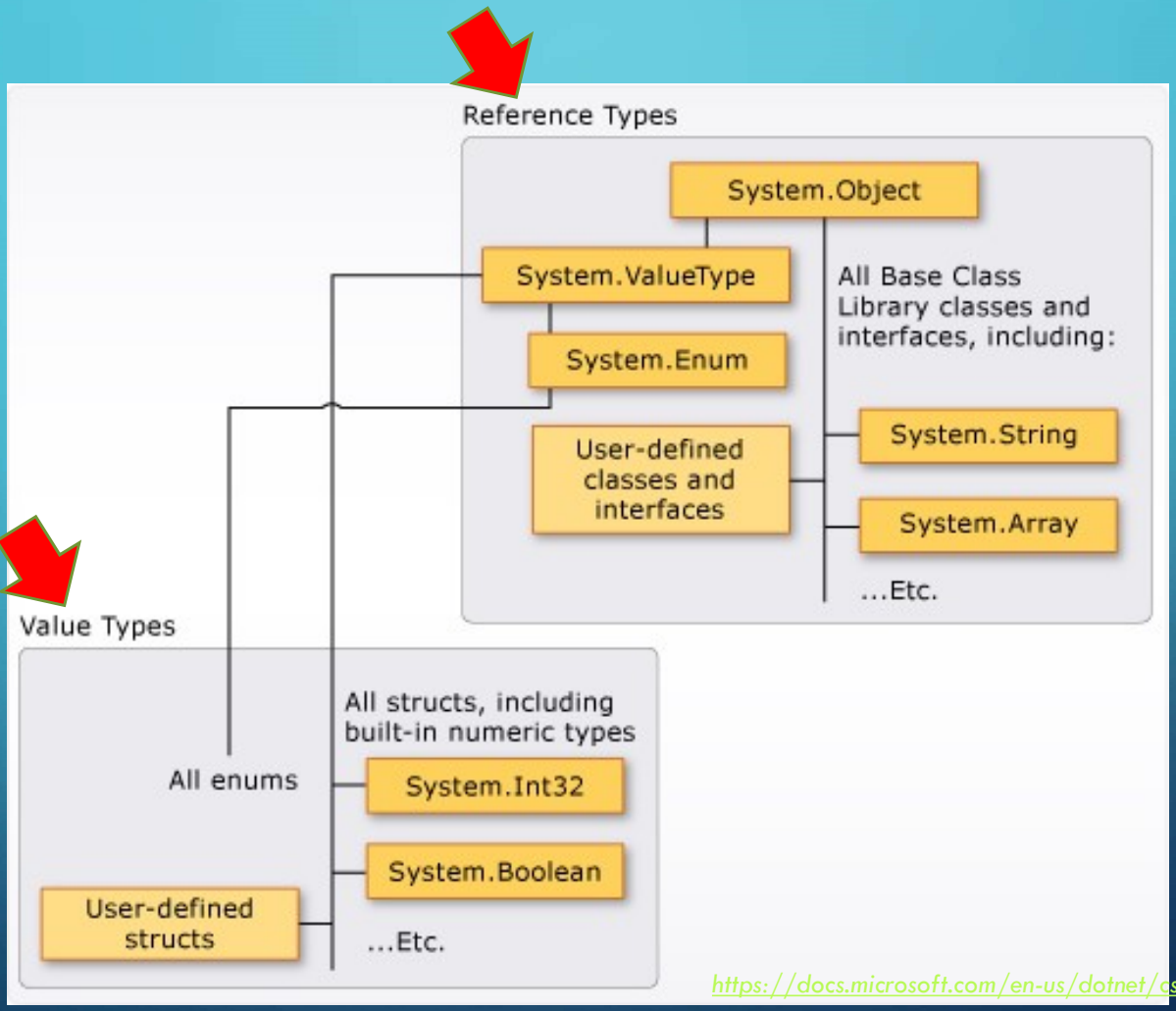
- Os Data Types em C# são utilizados para definir o tipo de dados que uma variável pode conter.
- Estão categorizados em dois tipos principais:

  * Tipos por Valor

  * Tipos por Referência

Nota: Na realidade, tecnicamente os tipos de dados não são do C# mas sim são do .Net

TIPOS DE DADOS EM C#



https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/types/

TIPO DE DADOS EM C#



Tipos por Valor

Os tipos por valor armazenam os seus dados diretamente:

int - Representa um número inteiro de 32 bits com sinal.

Exemplo: int quantidade = 100;

bool: *Representa um valor booleano (true or false).*

Exemple: bool hojeChove = true;

TIPO DE DADOS EM C#



float -Representa um número de ponto flutuante de precisão simples.

Exemplo: float preco = 7,5f;

double - Representa um número de ponto flutuante de precisão dupla.

Exemplo: double distância = 57,52;

char - Representa um único caractere Unicode.

Exemplo: char letra = 'A';

TIPO DE DADOS EM C#



Tipos de Referência

Os tipos de referência contêm referências aos seus dados:

string: Representa uma sequência de caracteres.

Exemplo: string nome = "Renato Miguel";

***object**: O tipo base do qual derivam todos os outros tipos. Exemplo: object obj = "Olá";*

OPERADORES EM C#



Operadores em C#

- Os operadores são símbolos que instruem o programa a executar operações sobre variáveis e valores.
- O C# inclui vários tipos de operadores:
  - ***Operadores Aritméticos***
    - *Utilizado para realizar operações matemáticas:*

OPERADORES EM C#

- ***Exemplos de Operadores Aritméticos:***

  +: ***Adição.***

  Exemplo: int soma = 5 + 3;

  -: ***Subtração.***

  Exemplo: diferença interna = 10 - 6;

  *: ***Multiplicação.***

  *Exemplo: int produto = 4 * 7;*

  /: ***Divisão.***

  Exemplo: quociente interno = 20/4;

  %: ***Módulo*** *(resto da divisão inteira).*

  Exemplo: int resto = 10% 3;

OPERADORES EM C#

Operadores Relacionais

- *Utilizados para comparar valores:*

== Igual a

Exemplo: bool isEqual = (5 == 5);

!= Diferente de

Exemplo: bool isNotEqual = (5! = 3);

> Maior que

Exemplo: bool éMaior = (10 > 5);

< Menor que

Exemplo: bool éLesser = (3 <7);

>= Maior ou igual a

Exemplo: bool isGreaterOrEqual = (5 >= 5);

<= Menor ou igual a

Exemplo: bool isLesserOrEqual = (2 <= 3);

=== Igual a, em valor e em tipo

OPERADORES EM C#

- *Operadores Lógicos*
  - *Utilizados para operações lógicas*

&& E lógico

Exemplo: resultado bool = (verdadeiro && falso);

|| OU lógico

Exemplo: resultado bool = (verdadeiro || falso);

! NÃO lógico

Exemplo: bool resultado = !true;

NAMESPACES EM C#

Namespaces em C#

Um **namespace** é um contentor que contém:

- classes,
- estruturas,
- enums,
- Delegates,
- Interfaces

Ajuda a organizar o código e evita conflitos de nomes.

NAMESPACES EM C#

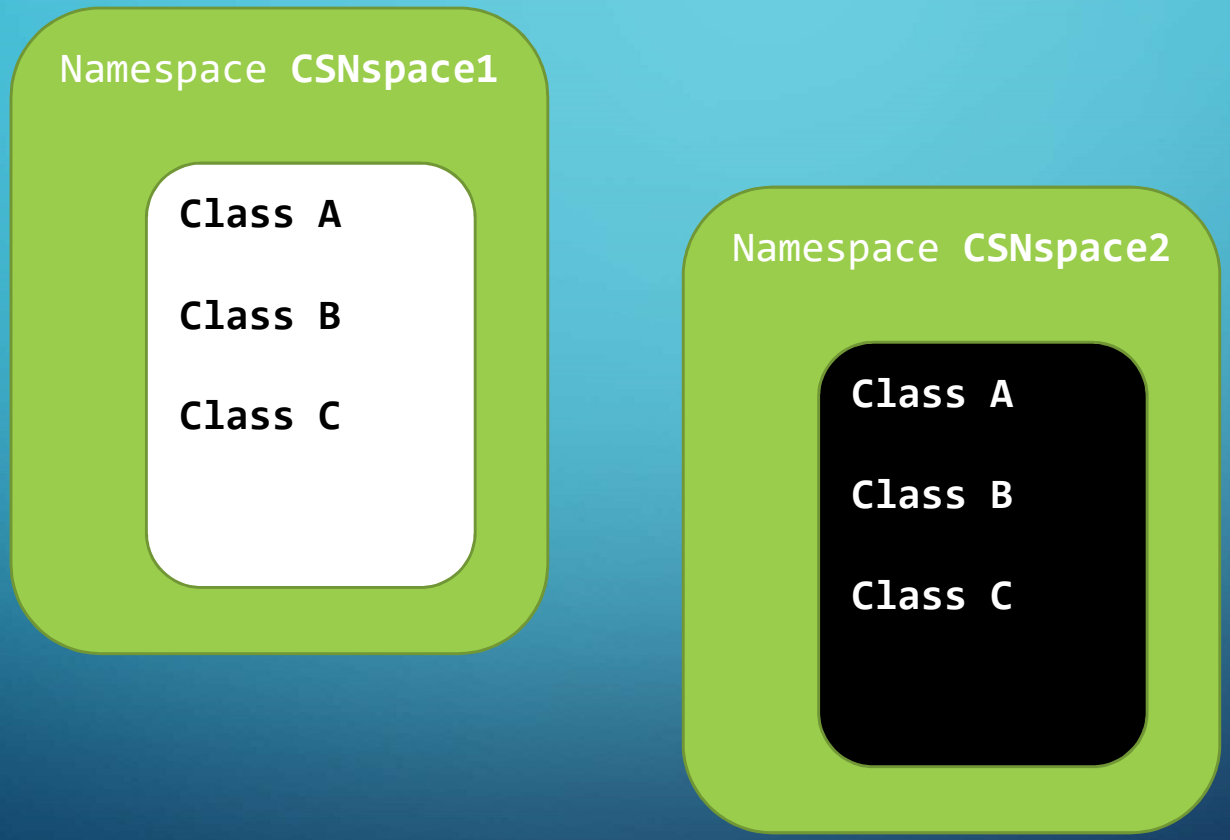
- O Código dos programas C# estão organizados em ***namespaces***
  - Organiza um conjunto de classes relacionadas entre si
  - As classes podem ter nomes iguais, desde que estejam em *namespaces* diferentes
  - Um *namespace* pode conter outros *namespaces*

NAMESPACES EM C#

Exemplo de Namespace em C#

```
namespace Models;
    class Aluno
    {
            public int Id {get; set;}
            public string Nome {get; set;}
            public string Mail {get; set;}
    }
```

C#: NAMESPACES




```
CSNspace1.A ...
CSNspace2.A ...
```

ENUMERAÇÕES EM C#

Enumumerações – enum

Uma enum é um tipo por valor que define um conjunto de constantes nomeadas

Exemplo:

```
enum Dias
{
    Domingo,
    Segunda-feira,
    Terça-feira,
    Quarta-feira,
    Quinta-feira,
    Sexta-feira,
    Sábado
}
```

ESTRUTURAS EM C#

Estrutura

- Uma estrutura é um tipo por valor que contém data members e métodos

```
struct Ponto
{
    public int X;
    public int Y;
    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
    public void Display()
    {
        Console.WriteLine($“Coordenadas dos Pontos:  ({X}, {Y})");
    }
}
```

ESTRUTURAS EM C#



- ***Class*** é tipo *referência* ≠ ***Struct*** é tipo *valor*
- Estruturas podem conter construtores, constantes, métodos, propriedades, eventos,…
- Usa-se a ***keyword*** `struct`
- Pode ser inicializada com ou sem o ***new***
- Estruturas não podem incluir constructores ou destrutores *default*, mas podem ter construtores parametrizados

ESTRUTURAS EM C#



- Podem implementar interfaces
- Não podem herdar outra estrutura ou classe
  - Membros das estruturas não podem ser especificados como ***abstract***, ***virtual*** ou ***protected***.
  - Devem ser inicializadas com o ***new*** de forma a poder usar-se as suas propriedades, métodos ou eventos
- Um ***`struct`*** pode ser usado como um tipo que permite valor nulo e receber um valor nulo.



Variáveis

- As variáveis são utilizadas para armazenar dados

- Têm de ser declarados com um tipo de dados específico

Exemplo de declaração de Variável

```
int idade = 25;

string nome = “Renato";

bool isEstudante = true;
```



Literais

- Os literais são valores constantes atribuídos a variáveis.
- Podem ser de vários tipos, como;
  - literais inteiros,
  - literais de decimais,
  - literais de caracteres,
  - literais de string.

Exemplo de Literais

Literal de inteiros:

```
int number = 10;
```

Literal de decimais

```
float pi = 3.14f;
```

Literal de caracteres

```
char initial = 'A';
```

Literal de String:

```
string greeting = "Hello";
```

C#: ARRAYS



Numero[0]	Numero[1]	Numero[2]	Numero[3]	…

```
int[] tabela;
int[] numeros;
tabela = new int[10];
numeros = new int[20];
string[,] saladeaula;
saladeaula = new string[10, 6];
int[,,] cubo;
```

C#: ARRAYS

```
int[] numeros= new int[5] { 1, 2, 3, 4, 5 };
string[] nomes= new string[3] { "Joana", "Filipa", "José" };

// ou

int[] numeros = new int[] { 1, 2, 3, 4, 5 };
string[] nomes = new string[] {"Joana", "Filipa", "José" };

// ou

int[] numeros = { 1, 2, 3, 4, 5 };
string[] nomes = {"Joana", "Filipa", "José" };
```

C#: *ARRAYS* – ACESSO AOS ELEMENTOS



```
int[] n = new int[10];
int i, j;

for (i = 0; i < 10; i++)
{
    n[i] = i + 100;
}


for (j = 0; j < 10; j++)
{
    Console.WriteLine("Elemento[{0}] = {1}", j, n[j]);
}
```

C#: ARRAYS – ACESSO AOS ELEMENTOS

- *foreach*

```
int[] n = new int[10];

for (int i = 0; i < 10; i++)
{
    n[i] = i + 100;
}

foreach (int j in n)
{
    int i = j - 100;
    Console.WriteLine("Elemento[{0}] = {1}", i, j);
}
Console.ReadKey();
```

BOXING E UNBOXING EM C#

Boxing

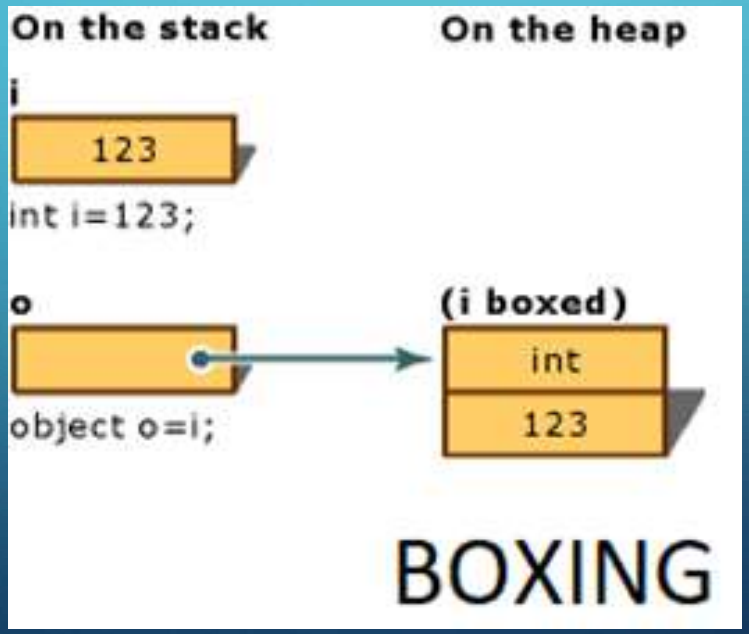
- O ***boxing*** é o processo de conversão de um tipo de valor num tipo de objeto ou em qualquer tipo de interface implementado por esse tipo de valor.

- Quando um tipo de valor é colocado em "*boxed*", é agrupado dentro de um objeto e armazenado no *heap*

BOXING E UNBOXING EM C#

Exemplo de Boxing

```
int i = 123; // Tipo Valor
object obj = i; // Boxing
```

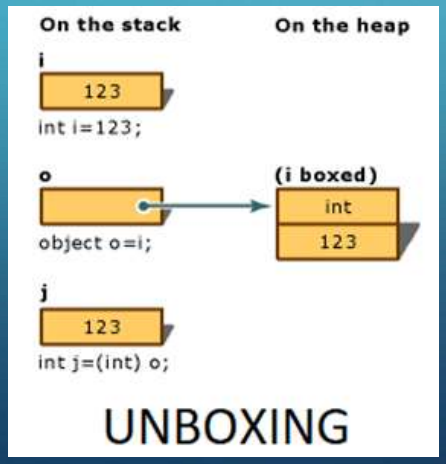


BOXING E UNBOXING EM C#

Unboxing

- O *unboxing* é o processo de conversão de um tipo de objeto novamente para um tipo de valor.
- Envolve extrair o tipo de valor do objeto

```
object obj = 123; // Objecto Boxed
int num = (int)obj; // Unboxing
```



MANAGED E UNMANAGED CODE EM C#

Managed Code - Código gerido

- O *Managed Code* é executado pelo Common Language Runtime (CLR) em .NET

- O CLR fornece vários serviços, como o Garbage Colector, o tratamento de Exceções e a segurança de tipo, que simplificam o desenvolvimento e aumentam a segurança.



Unmanaged Code - Código não gerido

- *O Unmanaged Code é executado diretamente pelo sistema operativo*
- *Está escrito em linguagens como C ou C++ e requer uma gestão explícita de memória.*



Em Resumo – Managed e Unmanaged Code

- O **Managed Code** *é executado sob o controlo do CLR, proporcionando uma gestão automática da memória*

- O **Unmanaged Code** *é executado diretamente no sistema operativo, exigindo uma gestão explícita da memória*



Type Casting em C#

A conversão de tipos é o processo de conversão de um tipo para outro.

- Em C#, existem dois tipos principais de conversão:
  - Implícita
  - Explícita.

TYPE CASTING EM C#

Implicit casting - Conversão Implícita

- A conversão implícita é executada automaticamente pelo compilador C# e não requer qualquer sintaxe especial.
- Geralmente ocorre ao converter um tipo de dados "mais pequeno" num tipo de dados "maior"

```
int num = 123;
double doubleNum = num; // Implicit casting
```



Explicit casting - Conversão Explícita

- A conversão explícita requer um operador de conversão para converter um tipo noutro.
- É utilizado ao converter um tipo de dados maior num tipo de dados mais pequeno ou ao converter entre tipos "incompatíveis"

```
double doubleNum = 123.45;
int num = (int)doubleNum; // Explicit casting
```

TYPE CASTING EM C#

Métodos para Conversão de Tipos

- O C# também fornece métodos para a conversão de tipos que lidam com conversões complexas, geralmente com verificação de erros
- Exemplo

```
string strNum = "123";
int num;
if (int.TryParse(strNum, out num))
{
    Console.WriteLine($"Convertido em numero: {num}");
}
else
{
    Console.WriteLine("A Conversão falhou.");
}
```



Em Resumo – Métodos para Conversão de Tipos

- A conversão implícita é segura e automática.
- A conversão explícita requer sintaxe explícita e pode levar à perda de dados.
- Os métodos de conversão de tipo proporcionam conversões seguras e controladas.

EM RESUMO

- ***Reais***
  - Usado para valores do tipo ***float***, ***double*** e ***decimal***
  - Podem ter notação exponencial
  - O sufixo ‘f’ ou ‘F’ significa ***float***
  - O sufixo ‘d’ ou ‘D’ significa ***double***
  - O sufixo ‘m’ ou ‘M’ significa ***decimal***
  - A interpretação por omissão é double

```
float realNumber = 12.5f;
```

C#: STRING - *ESCAPE* CARATERES



String

Sequência de *Escape*	Carater produzido
\'	Pelica
\"	Aspas
\\	Barra invertida
\0	Null
\a	Alert (origina um *beep*)
\b	Backspace
\f	*Form feed*
\n	Mudança de linha
\r	*Carriage return*
\t	Tabulação horizontal
\v	Tabulação vertical

C#: STRING - *ESCAPE* CARATERES

- ***String*** (cont.)
  - Pode conter o prefixo @ (*verbatim strings literals*), permitindo reduzir as sequências de *espace*

```
string citacao= "\"Caros, Aula de PW!\", ....";
string caminho = "C:\\3Ano\\PW\\exemplo.exe";
citacao = @"""Caros, Aula de PW!"", ....";
caminho = @"C:\3Ano\PW\exemplo.exe";
string str = @"string exemplo";
```

Verbatim strings

TIPOS DE DADOS EM C#: EXEMPLO

```
int meuInteiro;
string minhaString;

meuInteiro = 30;
minhaString = "\"meuInteiro\" igual a ";

Console.WriteLine($"{minhaString} {meuInteiro}");
Console.ReadKey();
```

Atribuição de valores fixos
Valores literais

Escape carateres

Característica do C# a partir da versão 6
String Interpolation

```
"meuInteiro" igual a  30
```

PROGRAMAÇÃO ORIENTADA A OBJETOS



- Representação de cada elemento em termos de um objeto, ou classe.
- Aproximar o sistema que se cria com o que se observa no mundo real
- Reutilização de código
  - Temporal e número de linhas de código
  - Independência
- Facilidade de leitura e manutenção de código, …

POO – PRINCÍPIOS BÁSICOS

Programação Orientada a Objetos

Abstração

Encapsulamento

Herança

Polimorfismo

PROGRAMAÇÃO ORIENTADA A OBJECTOS



- Auxilia na solução de problemas aproximando o sistema que se cria com o que se observa do mundo real;
- Dada a complexidade do mundo real, é necessário abstrair conhecimento relevante e encapsular dentro de **objetos**;
- Conjunto de objetos trabalham juntos para realizar uma tarefa;

PROGRAMAÇÃO ORIENTADA A OBJECTOS



- O mundo é composto por diversos **objetos** que possuem um conjunto de **características** e um **comportamento** bem definido;
- Definir **abstrações** dos objetos reais existentes;
- Todo **objeto** possui as seguintes características:
  - *Estado*: conjunto de propriedades de um objeto;
  - *Comportamento*: conjunto de ações possíveis sobre o objeto;
  - *Unicidade*: todo objeto é único.

PROGRAMAÇÃO ORIENTADA A OBJECTOS

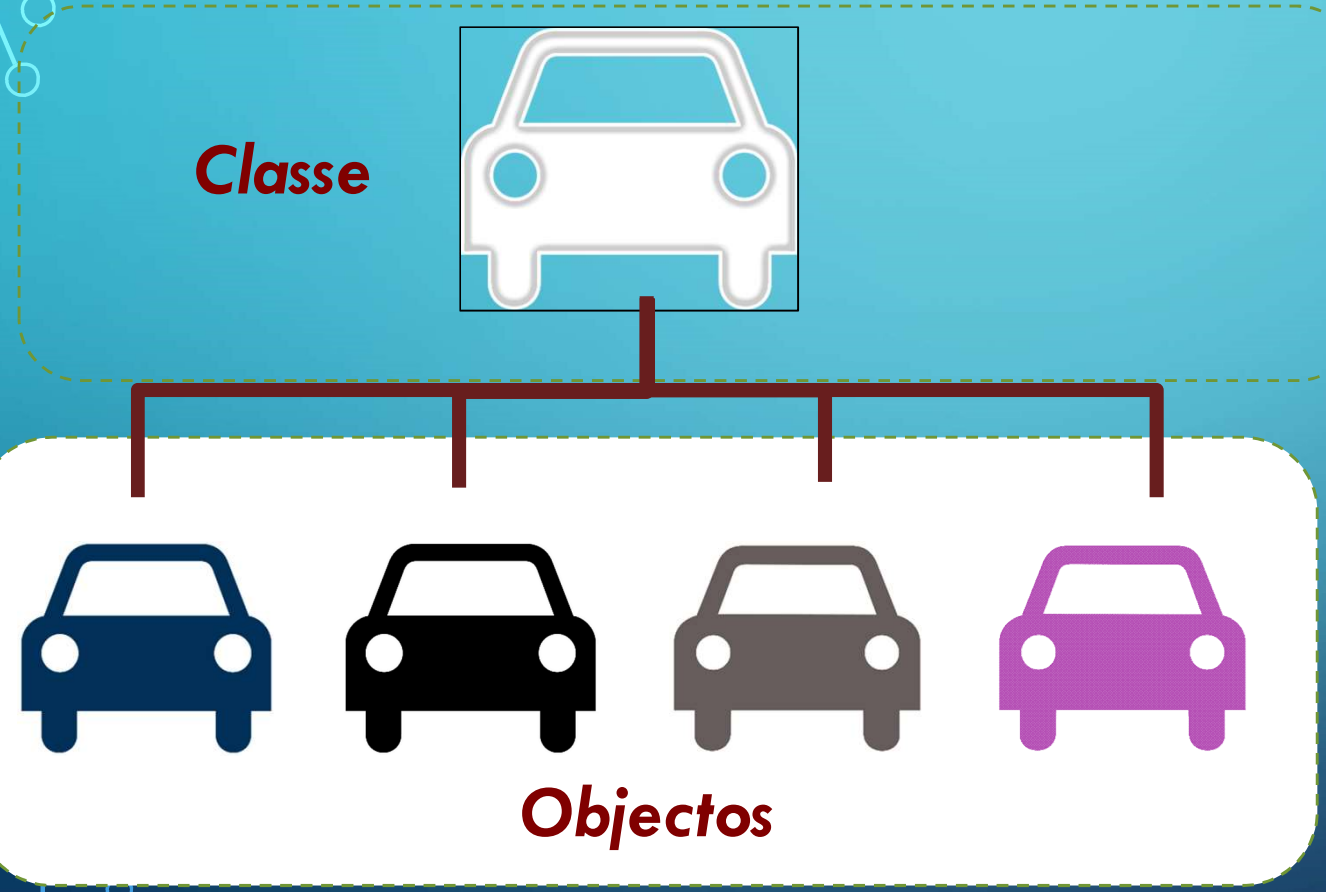
- A comunicação entre objectos é efetuada por mensagens

- **Objecto = Dados + Código**

- **Classes**
  - Definem os tipos de objectos
  - Definem quais os dados e o código dos objectos

***Dados* → atributos**
***Código* → métodos**

PROGRAMAÇÃO ORIENTADA A OBJECTOS



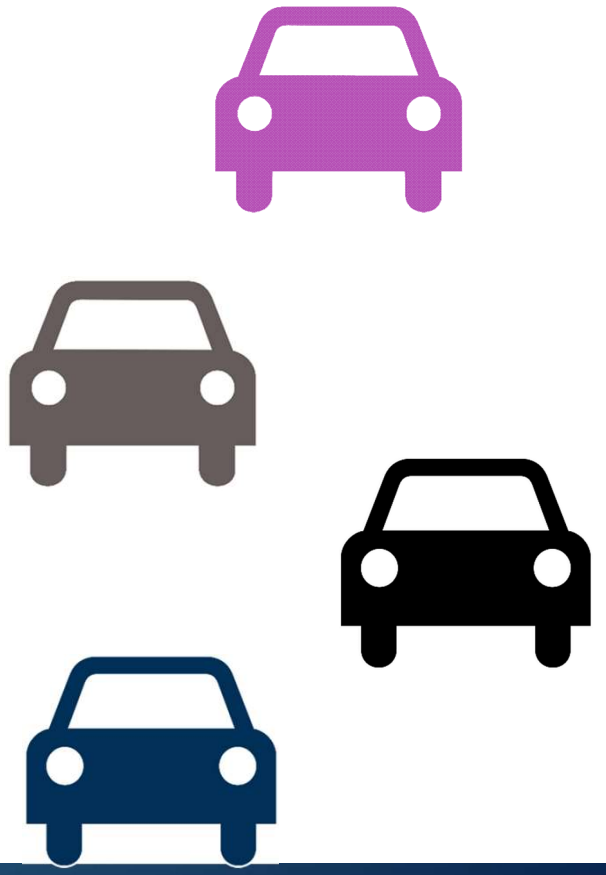
As **classes** são a caracterização abstrata de algo (cliente, empregado, carro,…)

Objeto ≠ Classe

Objeto = Instância de uma Classe

PROGRAMAÇÃO ORIENTADA A OBJECTOS

```
public class Carro
{
    private string cor;
    private string modelo;
    private string anoFabrico;
    private string combustivel;
    public void Anda()
    {   ...
    }
    public void Para()
    {   ...
    }
    public void Acelera()
    {   ...
    }
}
```



C#: OBJETOS

- Para criação de um objeto:
  - **Definição da Classe**
    - Considerada como um *template* do objeto
    - Exemplo: *Template* de um Carro
  - **Criação do objeto**
    - Objetos de uma classe criados em *runtime*
    - Exemplo: Carro em si! Instância do Carro

C#: CLASSES

- Sintaxe geral:

```
acesso class nome_da_classe
    {
        // Bloco de Instruções
    }
```

  - acesso
    - nível de proteção da classe (*internal* / *public*)
    - Por omissão, assume o nível *internal*
    - Pode ser também *abstract* ou *sealed*

C#: CLASSES

```
using System;
namespace Exemplo {
    class Program {
        static void Main(string[] args)
        {
            @override x = new @override();
            Console.WriteLine(x.Numero);
            Console.ReadKey();
        }
    }
    class @override
    {
        public int Numero { get; set; }

        public @override()
        {
            Numero = 5;
        }
    }}
```

C#: CLASSES

- *Construtor*
  - Se não for definido um construtor para uma classe, o C# define um por omissão

```
class MinhaClasse {
    public MinhaClasse()
        {
            // Construtor Default
        }

    public MinhaClasse(int meuInt)
        {
            // Construtor com parâmetro
        }
}
```

C#: CLASSES

```
class Motociclo {
    public string nomeCondutor;
    public int velocidade;
    public Motociclo() {}
    public Motociclo(int velocidade): this(velocidade, "") { }
    public Motociclo(string nome) : this(0,nome) { }
    public Motociclo(int velocidade, string nome) {
        if(velocidade>20) { velocidade = 20; }
        this.velocidade = velocidade;
        nomeCondutor = nome;
    }
}
```

C#: CLASSES

- *Destrutor*

```
class MinhaClasse
{
   ~MinhaClasse()
      {
         //Destrutor
      }
}
```

- Executado quando o *Garbage Collector* ocorre, permitindo libertar recursos

C#: CLASS

Campos

Método (Função)

```
using System;
1 reference
public class MyClass
{
    public string  myField = string.Empty;
    0 references
    public MyClass()
    {
    }
    0 references
    public void MyMethod(int parameter1, string parameter2)
    {
        Console.WriteLine("First Parameter {0}, second parameter {1}", parameter1, parameter2);
    }

    0 references
    public int MyAutoImplementedProperty { get; set; }

    private int myPropertyVar;

    0 references
    public int MyProperty
    {
        get { return myPropertyVar; }
        set { myPropertyVar = value; }

    }
}
```

Nome da Classe

Construtor

Propriedade *Auto-Implemented*

Propriedade



C#: MEMBROS DE UMA CLASSE

- Definindo uma classe, fornece-se definições para todos os membros:
  - Variáveis onde são armazenados os dados, designados como ***campos*** ou ***atributos*** *(fields)*;
  - **Métodos** que permitem manipular os dados;
  - **Propriedades**
    - Recorrendo aos *getters* e *setters*



C#: ATRIBUTOS / CAMPOS

- *Atributos* são também designados como *campos* (*member fields*)
- Permitem definir as características/dados que especificam os objetos
- Em C# os atributos de uma classe são definidos no interior da mesma
- Sintaxe:
  - `acesso`: Nível de proteção do atributo
    private (por omissão), *public*, *protected*, *internal*, *protected internal*

```
acesso tipo field;
```

C#: CAMPOS

Nível de proteção	Descrição
private **(por omissão)**	Só pode ser acedido a partir da classe onde foi declarado
public	Pode ser acedido externamente por outras classes ou métodos (incluindo fora do assembly onde foi definido)
protected	Só pode ser acedido a partir da classe onde foi declarado e a partir de classes derivadas
internal	Pode ser acedido a partir de qualquer classe (exceto fora do assembly onde foi definido
protected internal	Só pode ser acedido a partir da classe onde foi declarado ou partir de classes derivadas (exceto fora do assembly onde foi definido)

C#: NÍVEIS DE PROTEÇÃO DAS CLASSES

Nível de Proteção	Descrição
internal (por omissão)	Classe acessível apenas no projeto corrente
public	Classe acessível de qualquer parte
abstract **internal abstract**	Acessível apenas no projeto corrente e não pode ser instanciada, apenas derivada de
public abstract	Pode ser acessível de qualquer parte e não pode ser instanciada, apenas derivada de
sealed **internal sealed**	Acessível apenas no projeto corrente e não pode ser derivada de, apenas instanciada
public sealed	Acessível de qualquer parte e não pode ser derivada de, apenas instanciada

C#: *PARTIAL CLASS*

- Pode-se "partir" a classe em vários ficheiros
  - O nome dos ficheiros pode ser qualquer um, importante é definir o tipo (neste caso classe) com o mesmo nome
- Recurso à keyword `partial`
- *Visual Studio* recorre constantemente a classes parciais
- Por exemplo:
  - Métodos e Propriedades num ficheiro e restantes elementos noutro…

C#: PARTIAL CLASS

```
namespace PartialClasse
{
    partial class MinhaClasse
    {
        // Campos e Construtores
        private int idade;
        public MinhaClasse()
        {
            idade = 0;
        }
        public MinhaClasse(int i)
        {
            idade = i;
        }
    }
}
```

MinhaClasse.cs

```
namespace PartialClasse
{
    partial class MinhaClasse
    {
        // Metodos e Propriedades
        public int Idade
        {
            get { return idade; }
            set  {
                if (value<0||value>100)
                    idade = 0;
                else idade = value;
            }
        }
    }
}
```

MinhaClasse.Core.cs

C#: ENCAPSULAMENTO

- **Encapsulamento** tem como objetivo a ocultação de pormenores internos da classe
- Fornecida uma *interface pública* constituída por métodos que são a única coisa que o resto do programa se apercebe da classe
- A implementação da classe pode ser modificada sem que isso interfira com o resto do programa, desde que não se altere a interface



POO: ENCAPSULAMENTO

- Adicionam segurança na implementação em OO uma vez que escondem elementos internos, criando uma espécie de *black box*
- Maior parte das linguagens OO implementam o encapsulamento com recurso a métodos especiais chamados de *getters* e *setters*
- Em C# as **propriedades** evitam o acesso direto aos campos do objeto

C#: PROPRIEDADES

- Definidas de forma semelhante aos campos, permitindo desempenhar um processamento adicional
  - Recurso às *keywords* *get* e *set*
- Estrutura básica:

```
public int MinhaPropriedade
    {
        get {
       // código para o get
        }
        set {
       // código para o set
        }
    }
```

C#: PROPRIEDADES

- Propriedade
  - Encapsula um campo privado
  - Permite especificar o *get* para retribuir valores de um campo e permite especificar o *set* para definir um valor
  - *Exemplo:*

```
private int meuCampo;
public int MeuCampo
    {
        get { return meuCampo;
        }
        set { meuCampo=value;
        }
    }
```

C#: PROPRIEDADES

- É possível também adicionar lógica nas propriedades:

```
class Exemplo
{
    private int propriedade;
    public int Propriedade {
        get
        {
            return propriedade / 2;
        }
        set
        {
            if (value > 100) propriedade = 100;
            else propriedade = value;
        }
    }
}
```

Saída?

```
Exemplo x = new Exemplo();
c.Propriedade = 1;
```

C#: PROPRIEDADES

```
using System;
class IntervaloTempo
{
    private double segundos;

    public double Horas
    {
        get { return segundos / 3600; }
        set {
            if (value < 0 || value > 24)
                throw new ArgumentOutOfRangeException(
                      $"{nameof(value)} deve estar compreendido entre 0 e 24.");
            segundos = value * 3600;
        }
    }
}
```

```
IntervaloTempo t = new IntervaloTempo();
t.Horas = 24;
 Console.WriteLine($"Tempo em horas: {t.Horas}");
```

C#: *AUTO IMPLEMENTED PROPERTIES*



- *Auto implemented Properties* em C#
  - Surgiram com o C# 3.0
  - Permitem uma declaração simplificada, quando não é necessária especificar qualquer lógica no *get* ou *set*
  - Não foram declaradas variáveis internas?
    - Criadas pelo compilador! Poupa tempo e torna o código mais limpo.

```
public String MeuNome
    {
        get; set;
    }
```

C#: AUTO IMPLEMENTED PROPERTIES

```
using System;
public class Aluno
{
    public string Nome  { get; set; }
    public decimal Idade { get; set; }
}
class Program
{
    static void Main(string[] args)
    {
        var aluno = new Aluno{ Nome = "xpto", Idade = 17 };
        Console.WriteLine($"{aluno.Nome} tem {aluno.Idade} anos!");
    }
}
```

C#: PROPRIEDADES

- Apenas de leitura
```
public int Numero {get;}
```

- Apenas de escrita
```
public int Numero {set;}
```



C#: PROPERTIES

```
public class Aluno
{
    string nome;
    decimal idade;
    public Aluno(string nome, decimal idade)
    {
        this.nome = nome;
        this.idade = idade;
    }
    public string Nome
    {
        get => nome;
        set => nome = value;
        }
    public decimal Idade
    {
        get => idade;
        set => idade = value;
        }
    }
}
```

C# 7.0

Expression-bodied members (EBM's)

O C# 6 permitiu a implementação de propriedades somente leitura e métodos através das expressões lambdas e isso foi chamado de EBM.

Na versão 7.0, este recurso foi melhorado permitindo a utilização de expressões lambdas em construtores, propriedades com get/set e finalizadores.

O objetivo do recurso é tornar o código conciso e legível fazendo com que o membro do tipo (construtor, métodos, propriedades, destrutor, etc) seja definido em uma única expressão.

```
var aluno = new Aluno{ Nome = "xpto", Idade = 17 };
Console.WriteLine($"{aluno.Nome} tem {aluno.Idade} anos!");
```

C#: MÉTODOS

- Ações que os objetos podem desempenhar, definidos dentro do corpo de uma classe
- Tecnicamente conhecidas por funções de membro de uma classe
  - Sintaxe: `acesso tipo nome_método(argumentos)`
  - Exemplo:

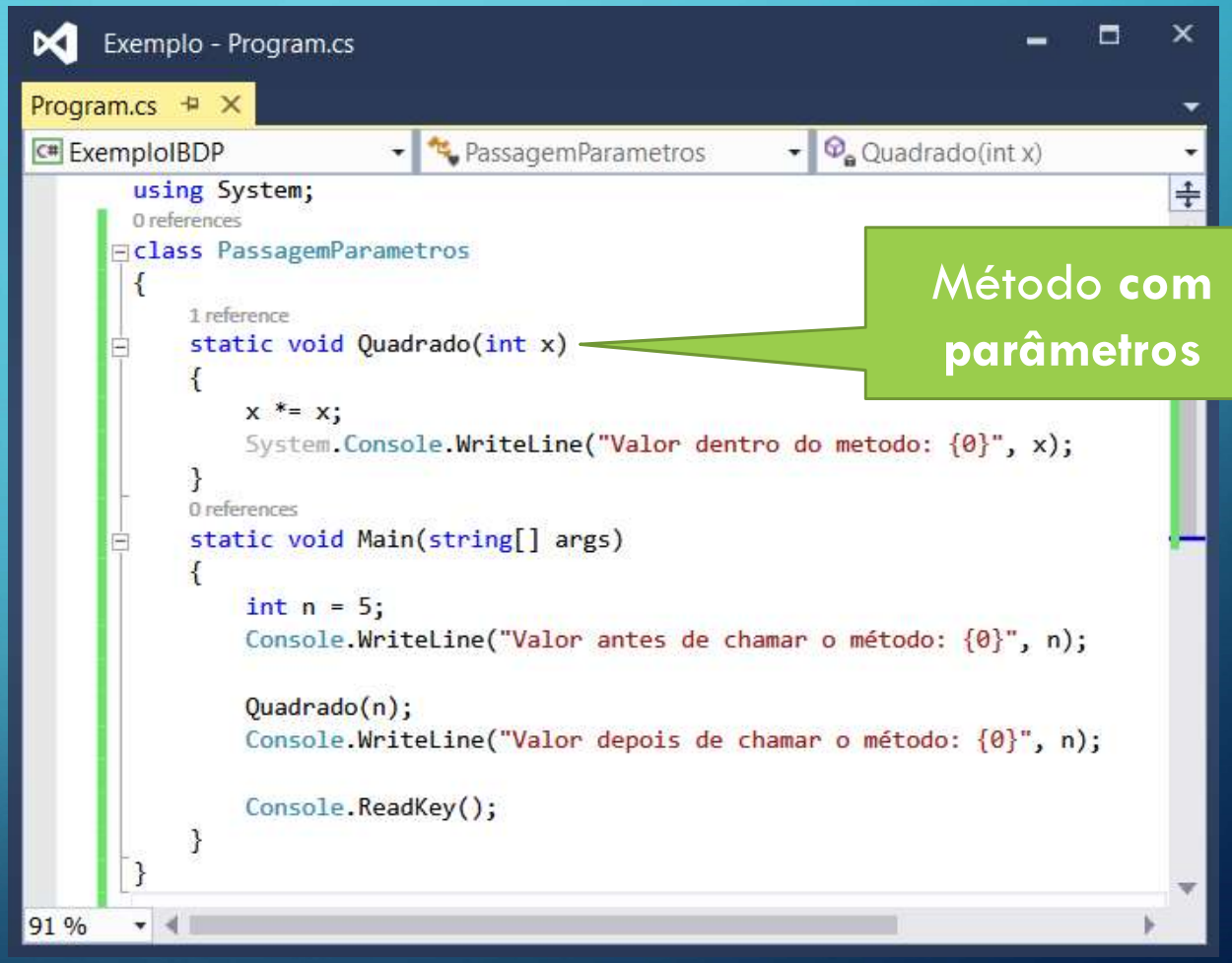
```
public void fazQualquerCoisa() { }

public String ObtemNome() { }
```

- Existem métodos próprios: construtores, …

C#: MÉTODOS - PARÂMETROS

- Por valor
- Por referência
  - *ref*



C#: PASSAGEM DE PARÂMETROS POR VALOR



- Por valor
- Por referência
  - *ref*

```
using System;
0 references
class PassagemParametros
{
    1 reference
    static void Quadrado(int x)
    {
        x *= x;
        System.Console.WriteLine("Valor dentro do metodo: {0}", x);
    }
    0 references
    static void Main(string[] args)
    {
        int n = 5;
        Console.WriteLine("Valor antes de chamar o método: {0}", n);

        Quadrado(n);
        Console.WriteLine("Valor depois de chamar o método: {0}", n);

        Console.ReadKey();
    }
}
```

C#: PASSAGEM DE PARÂMETROS POR VALOR



- Por valor
- Por referência
  - *ref*

```
using System;
0 references
class PassagemParametros
{
    1 reference
    static void Quadrado(int x)
    {
        x *= x;
        System.Console.WriteLine("Valor dentro do metodo: {0}", x);
    }
    0 references
    static void Main(string[] args)
    {
        int n = 5;
        Console.WriteLine("Valor antes de chamar o método: {0}", n);

        Quadrado(n);
        Console.WriteLine("Valor depois de chamar o método: {0}", n);

        Console.ReadKey();
    }
}
```

C#: PASSAGEM DE PARÂMETROS *REF*

- Valores
- Referência
  - *ref*

```
using System;
0 references
class PassagemParametros
{
    1 reference
    static void Quadrado(ref int x)
    {
        x *= x;
        System.Console.WriteLine("Valor dentro do metodo: {0}", x);
    }
    0 references
    static void Main(string[] args)
    {
        int n = 5;
        Console.WriteLine("Valor antes de chamar o método: {0}", n);

        Quadrado(ref n);
        Console.WriteLine("Valor depois de chamar o método: {0}", n);

        Console.ReadKey();
    }
}
```

- Devem ser inicializados antes de serem passados ao método

C#: PASSAGEM DE PARÂMETROS POR REFERÊNCIA



- Valores
- Referência
  - *ref*

Saída?

```
using System;
0 references
class PassagemParametros
{
    1 reference
    static void Quadrado(ref int x)
    {
        x *= x;
        System.Console.WriteLine("Valor dentro do metodo: {0}", x);
    }
    0 references
    static void Main(string[] args)
    {
        int n = 5;
        Console.WriteLine("Valor antes de chamar o método: {0}", n);

        Quadrado(ref n);
        Console.WriteLine("Valor depois de chamar o método: {0}", n);

        Console.ReadKey();
    }
```

```
Valor antes de chamar o método: 5
Valor dentro do metodo: 25
Valor depois de chamar o método: 25
```

C#: MÉTODOS – PARÂMETROS *OUT*

- **out**
  - Não necessita ser inicializado antes de ser passado ao método
  - (não faria sentido)

```
using System;
0 references
class Parametros
{

    1 reference
    static int MaxValor(int[] intArray, out int maxIndice)
    {
        int maxVal = intArray[0];
        maxIndice = 0;
        for (int i = 1; i < intArray.Length; i++)
        {
            if (intArray[i] > maxVal)
            {
                maxVal = intArray[i];
                maxIndice = i;
            }
        }
        return maxVal;
    }
    0 references
    static void Main(string[] args)
    {
        int[] meuArray = { 1, 8, 3, 6, 2, 5, 9, 3, 0, 2 };
        int maxIndice;
        Console.WriteLine($"O valor máximo no meuArray é { MaxValor(meuArray, out maxIndice)}");
        Console.WriteLine($"A primeira ocorrencia deste valor está no elemento { maxIndice + 1}");

        Console.ReadKey();
    }
}
```

C#: MÉTODOS – PARÂMETROS *OUT*

- **out**

```
using System;
0 references
class Parametros
{

    1 reference
    static int MaxValor(int[] intArray, out int maxIndice)
    {
        int maxVal = intArray[0];
        maxIndice = 0;
        for (int i = 1; i < intArray.Length; i++)
        {
            if (intArray[i] > maxVal)
            {
                maxVal = intArray[i];
                maxIndice = i;
            }
        }
        return maxVal;
    }
    0 references
    static void Main(string[] args)
    {
        int[] meuArray = { 1, 8, 3, 6, 2, 5, 9, 3, 0, 2 };
        int maxIndice;
        Console.WriteLine($"O valor máximo no meuArray é { MaxValor(meuArray, out maxIndice)}");
        Console.WriteLine($"A primeira ocorrencia deste valor está no elemento { maxIndice + 1}");

        Console.ReadKey();
    }
}
```

```
O valor máximo no meuArray é 9
A primeira ocorrencia deste valor está no elemento 7
```

C#: PARÂMETROS *REF* VS *OUT*

```
using System;
namespace RefVSOut {
    class Program
    {
        static void Main(string[] args)
        {
            int val1 = 0;
            int val2;

            MethodRef(ref val1);
            Console.WriteLine(val1);

            MethodOut( out val2);
            Console.WriteLine(val2);
        }
    ...
```

```
static void MethodRef(ref int value)
 {
            value = 1;
 }
static void MethodOut(out int value)
  {
     value = 2;
  }
```



C#: *PARAMS ARRAYS*



- C# suporta ***parameter arrays***
  - Permite a passagem de um número variável de parâmetros de tipo idêntico (ou classes relacionadas por herança) como um único parâmetro lógico
- Argumentos especificados com a *keyword params* podem ser processados, se chamado através do envio de um *array* fortemente "tipado" ou uma lista de itens separados por vírgula

C#: PARAMS ARRAYS



```
using System;
namespace ArrayApplication {
    class ParamArray {
        public int AddElements(params int[] arr) {
            int sum = 0;
            foreach (int i in arr)
                sum += i;
            return sum;
        }
    }
    class TestClass {
        static void Main(string[] args) {
            ParamArray app = new ParamArray();
            int sum = app.AddElements(512, 720, 250, 567, 889);
            Console.WriteLine("Soma: {0}", sum);
            Console.ReadKey();
        }
    }
}
```

C#: PARAMS ARRAYS

```
using System;
namespace ArrayApplication {
    class ParamArray {
        public int AddElements(params int[] arr) {
            int sum = 0;
            foreach (int i in arr)
                sum += i;
            return sum;
        }
    }
    class TestClass {
        static void Main(string[] args) {
            ParamArray app = new ParamArray();
            int sum = app.AddElements(512, 720, 250, 567, 889);
            Console.WriteLine(AddElements();
            Console.ReadKey();
        }
    }
}
```

C#: PARAMS ARRAYS

```
using System;
namespace ArrayApplication {
    class ParamArray {
        public int AddElements(params int[] arr) {
            int sum = 0;
            foreach (int i in arr)
                sum += i;
            return sum;
        }
    }
    class TestClass {
        static void Main(string[] args) {
            ParamArray app = new ParamArray();
            int sum = app.AddElements(int[] dados={1, 4, 5, 6 };
            Console.WriteLine(AddElements(dados);
            Console.ReadKey();
        }
    }
}
```

C#: PARÂMETROS OPCIONAIS



- O C# permite criar métodos que passam **argumentos opcionais**
- Permite invocação do método omitindo alguns argumentos "desnecessários"

C#: PARÂMETROS OPCIONAIS - EXEMPLO



```
class Program {
    static void Main() {
        // Omite os parâmetros opcionais
        Metodo();
        // Omite segundo parametro
        Metodo(4);
        // Sintaxe clássica
        Metodo(4, "Jose");
        // ERRO:  Metodo("Filomena");  Necessário especificar o nome do parâmetro
        Metodo(nome: "Filipa");
        // Especifica ambos os parâmetros
        Metodo(nome: "Maria“, numero: 5);
    }
    static void Metodo(int numero = 0, string nome = "Indefinido") {
        System.Console.WriteLine("Numero = {0}, Nome = {1}", numero, nome);
    }
}
```

C#: PARÂMETROS OPCIONAIS



```
 static void Metodo(int dia = System.DateTime.Now.Day ,
        string nome = "Indefinido")
{

    System.Console.WriteLine("Dia = {0}, Nome = {1}",
            dia, nome);
}
```



Programação Web

C#: Polimorfismo, Herança, …

Departamento de Engenharia Informática e de Sistemas
Instituto Superior de Engenharia de Coimbra/Instituto Politécnico de Coimbra



C#: POLIMORFISMO



- O **Polimorfismo** permite ao programador definir **métodos genéricos** em classes ou interfaces, que podem ser implementados de **diversas formas** nas respetivas subclasses
- Polimorfismo aplica-se apenas aos métodos da classe e, por definição, exige a utilização de herança
- Vantagens:
  - Reutilização de código;
  - Tempo de desenvolvimento;
  - Manutenção.

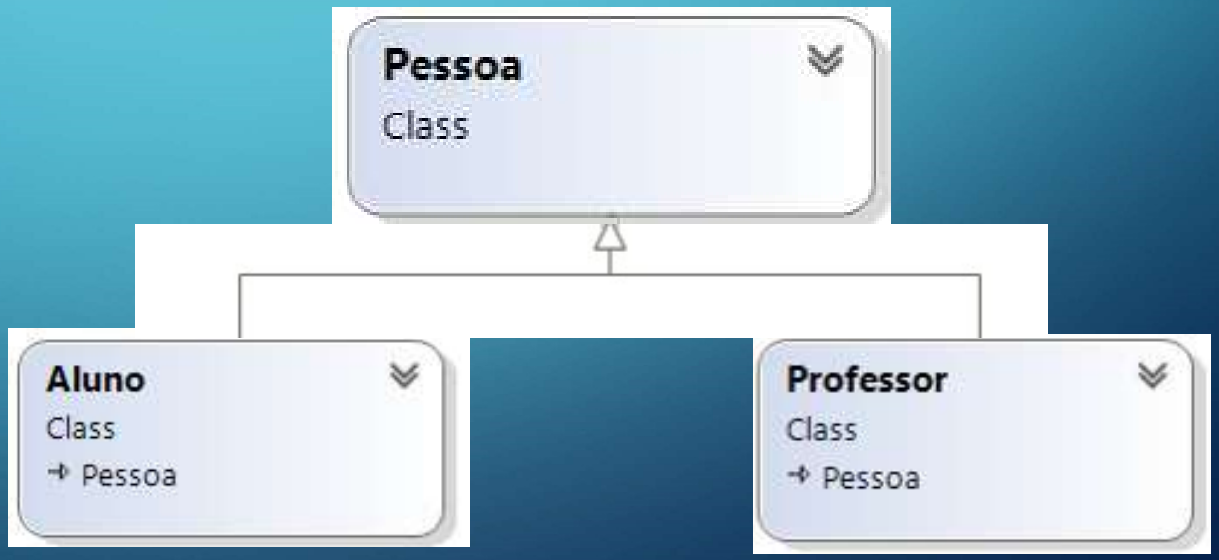
C#: HERANÇA



- A **Herança** é um dos principais conceitos da POO
- Permite reutilização de código e redução da complexidade
- Permite **construção de classes baseadas noutras classes** já existentes (neste caso, a primeira classe diz-se *classe base*)
- A subclasse:
  - Herda todas as propriedades e métodos da classe existente
  - Pode incluir ou sobrepor novas propriedades e novos métodos

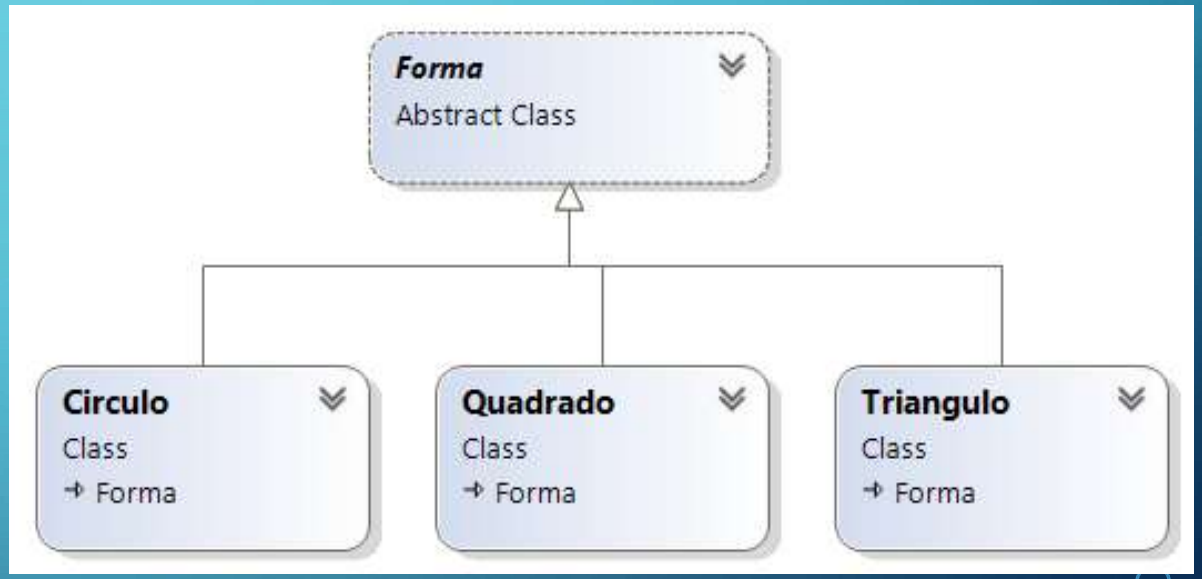
C#: HERANÇA

- Em C# apenas é permitido **uma classe base!**
- Mecanismo recursivo, permitindo criar uma hierarquia de classes
- Exemplo:

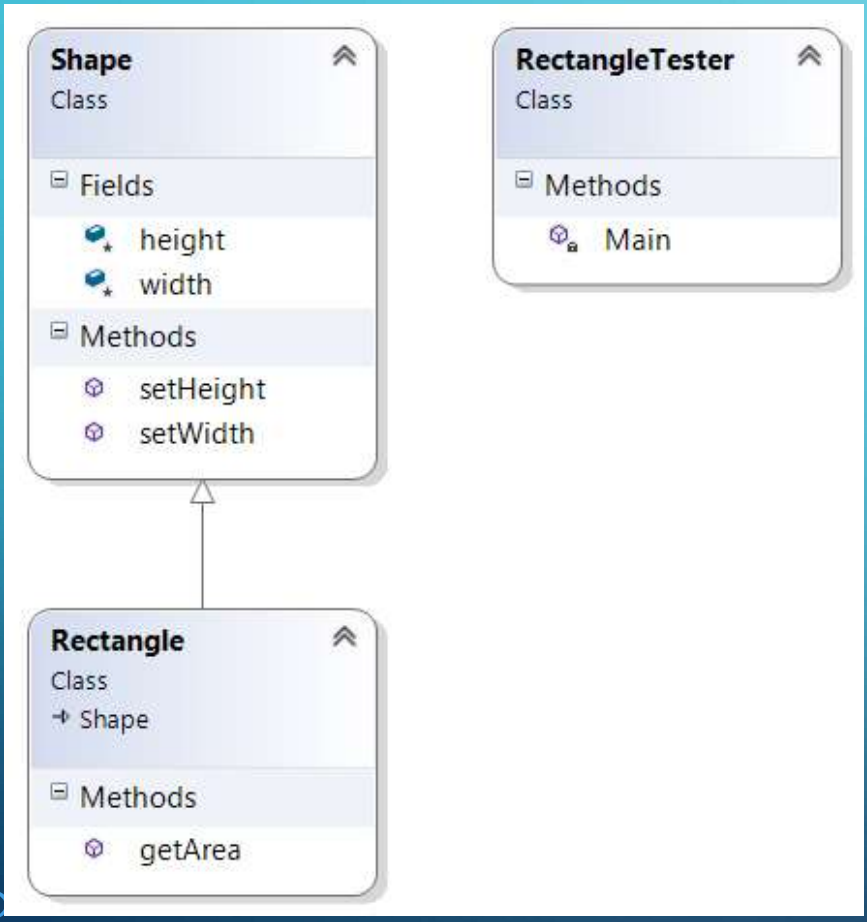


C#: HERANÇA (3)

- Exemplo 2:
  - Um “quadrado” é uma “forma”
  - Uma “forma” define uma propriedade comum “cor”
  - Um “quadrado” herda a propriedade “cor”

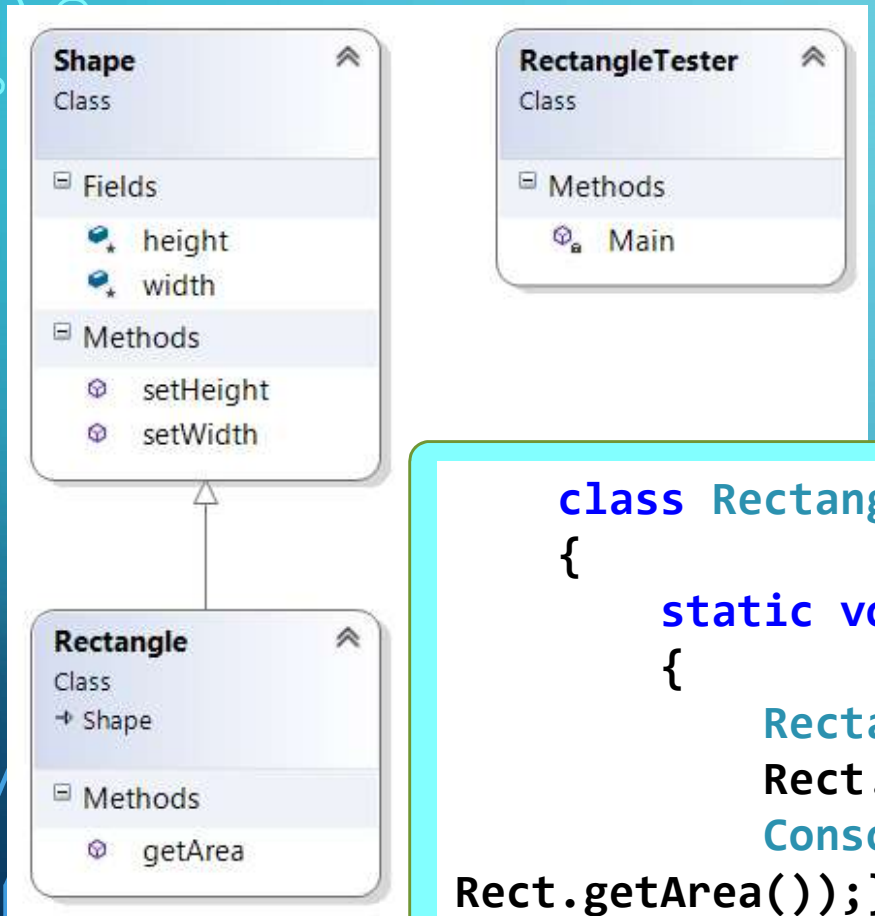


C#: HERANÇA – EXEMPLO (1)




```
class Shape
    {
        public void setWidth(int w)
        {
            width = w;
        }
        public void setHeight(int h)
        {
            height = h;
        }
        protected int width;
        protected int height;
    }
```

C#: HERANÇA – EXEMPLO (1)

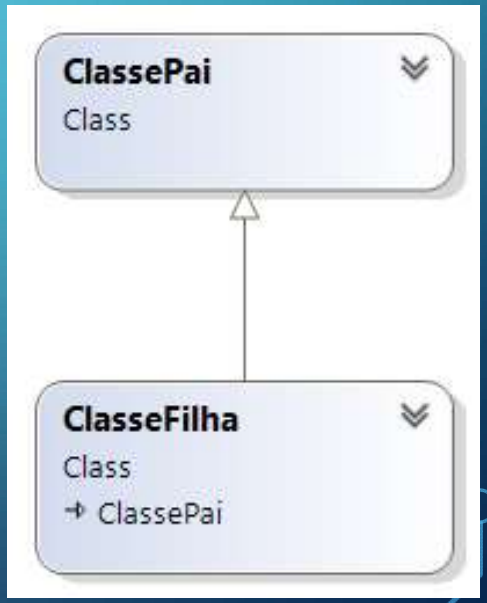



```
class Rectangle : Shape
   {
        public int getArea()
        {
            return (width * height);
        }
   }
```

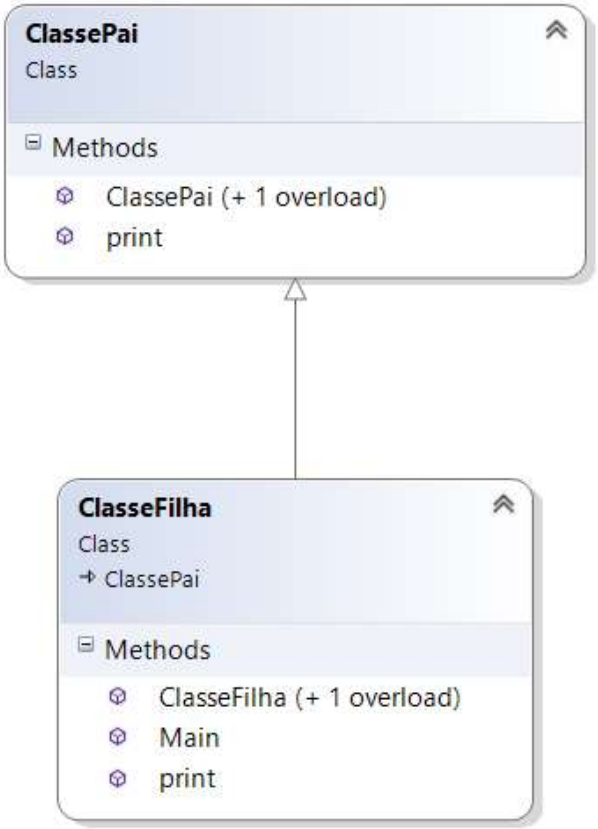
```
    class RectangleTester
    {
        static void Main(string[] args)
        {
            Rectangle Rect = new Rectangle();
            Rect.setWidth(5); Rect.setHeight(7);
            Console.WriteLine("Total area: {0}",
Rect.getArea());}
    }
```

C#: HERANÇA

```
public class ClassePai
{
    // Membros da classe
}
internal class ClasseFilha: ClassePai
{
    // Membros da classe
}
```



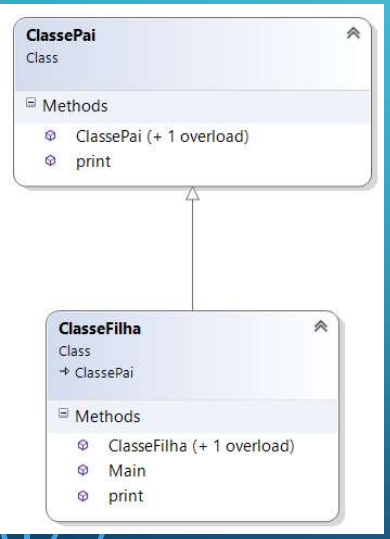
C#. HERANÇA




```
public class ClassePai
    {
        public ClassePai()
        {
            Console.WriteLine("Construtor Pai");
        }
        public ClassePai(string texto)
        {
            Console.WriteLine("Construtor Pai " + texto);
        }
        public void print()
        {
            Console.WriteLine("Print() da classe Pai.");
        }
    }
```

C#: HERANÇA

Ocultar qualquer implementação do método print() acima dele




```
public class ClasseFilha : ClassePai     {
        public ClasseFilha() : base("Mensagem Ola...")  {
            Console.WriteLine("Construtor Filha");
        }
        public ClasseFilha(string x) : base(x) {
            Console.WriteLine("Construtor Filha com string");
        }
        public new void print()  {
            base.print();
            Console.WriteLine("Print() da classe filha");
        }
        public static void Main()  {
            ClasseFilha filha = new ClasseFilha();
            ClasseFilha filha2 = new ClasseFilha("String Parametro");
            filha.print();
            ((ClassePai)filha).print();
    }
     }
```

C#: HERANÇA

- Membros da classe **base** com acesso ***private*** não são acessíveis da classe derivada
  - Nível de proteção ***protected*** permite que as classes derivadas tenham acesso ao membro, sem o tornar público
- Membros da classe base podem ser virtuais (***virtual***)
  - A classe que herda uma classe com membros virtuais, pode ser ***overridden*** pela classe que a herda -> i.e. altera o método original existente na classe pai
  - Isso, permite que a classe derivada possa ter uma implementação alternativa para o membro.

C#: CLASSES E MÉTODOS ABSTRATOS



- A Classe *Base* pode ser **Abstrata**
  - Uma Classe Abstrata é uma classe conceptual no qual se define funcionalidades para que as subclasses (que a herdam) possam implementá-las de forma **não obrigatória**
- Uma *classe abstrata* pode ser somente herdada e não instanciada
- Classes abstratas podem ter membros abstratos, os quais não tem implementação na classe base, portanto, tem de ser fornecida uma implementação na classe derivada caso sejam usados na classe derivada

C#: CLASSES E MÉTODOS ABSTRATOS(2)



- Quando uma classe possui pelo menos um método abstrato esta deve também ser declarada como abstrata
- Classes bases abstratas podem já fornecer implementação dos membros
- Uma classe abstrata pode herdar de outra classe abstrata

```
public abstract class nome_da_classe
    {
        // Membros da classe, podem ser abstratos
    }
```

C#: CLASSES E MÉTODOS ABSTRATOS(3)

- Um método abstrato é um método que ***não possui implementação*** na classe abstrata, apenas possui a definição da sua **assinatura**
- A sua implementação deve ser feita na **classe derivada**
- Os métodos podem ou não ser abstratos, mas quando estes são abstratos, a sua **implementação** é **obrigatória**!

C#: MEMBROS VIRTUAIS

- Membro ***virtual***
  - É um membro na classe base que define uma **implementação por omissão** que pode ser alterada (*overridden*) pela classe derivada
  - **Por contraste, um método abstrato,** é um membro na classe base que não fornece qualquer implementação por omissão, mas fornece a sua assinatura.

CLASSES ABSTRATAS E *SEALED*

- A palavra chave ***abstract*** permite criar classes e membros de classes incompletas que têm de ser implementados na classe derivada

- A palavra chave ***sealed*** evita que a classe ou membros virtuais sejam herdados

C#: *INTERFACES*

- Para a definição de ***Interfaces***, utiliza-se a palavra chave ***interface***

```
public interface IMeuInterface
{
    // Membros da Interface
}
```

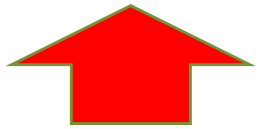
- Todas as interfaces são **públicas**
- A partir da versão do C# 9, os métodos declarados numa interface podem possuir implementação e são sempre públicos
- Quando uma interface é referenciada tem de se implementar a funcionalidade dos seus métodos

C#: HERANÇA E INTERFACES

- Herança de *Interfaces*

```
public class MyClass : IMeuInterface
{
    // Membros da class
}
```

```
public class MyClass : MyBase, IMeuInterface, IMeu2Interface
{
    // Membros da class
}
```



C#: INTERFACES

Implicitamente public e abstract

```
public interface IMeuInterface
{
        // Membros da Interface
        int método();
}
```

PALAVRA CHAVE *AS*

- Os *cast* explícitos são avaliados em *runtime* e não em tempo de compilação
- ***as*** tenta converter um objeto para um tipo específico e retorna ***null*** se a conversão falhar.

```
object meuObjecto = new Circulo(“CirculoObjecto");
    Quadrado quadrado = meuObjecto as Quadrado;
    if (quadrado==null)
    {
        Console.WriteLine("O objecto não é um quadrado...");
    }
```

PALAVRA-CHAVE *IS* EM C#

- ***is*** permite de forma rápida determinar se um determinado **objecto** é compatível com um **tipo** e devolve *false* caso não sejam

```
object meuObjecto = new Rectangulo(“RectanguloObjecto");
Quadrado quadrado = meuObjecto as Quadrado;
  if (quadrado is Frango)
  {
     Console.WriteLine("O objecto é um Rectangulo...");
  }
```



Programação Web

C#: Manipulação de Excepções

Departamento de Engenharia Informática e de Sistemas
Instituto Superior de Engenharia de Coimbra/Instituto Politécnico de Coimbra



C#: MANIPULAÇÃO DE EXCEPÇÕES



- *try*
- *catch*
- *finally*
- *throw*

```
try
{
    // instruções que podem causar excepção
}
catch( ExceptionName e1 )
{
    // Codigo que manipula a excepção
}
catch( ExceptionName e2 )
{
    // Codigo que manipula a excepção
}
catch( ExceptionName eN )
{
    // Codigo que manipula a excepção
}
finally
{
    // Codigo a ser executado
}
```

C#: MANIPULAÇÃO DE EXCEPÇÕES



- E o **Throw ?**
  - ***throw***
  - ***throw ex***

```
try
{
    //algum código
}
catch
{
    throw;
}
```

```
try
{
    //algum código
}
catch (Exception ex)
{
    throw ex;
}
```

- ***throw*** **-** quando se utiliza o ***throw*** passamos a mesma excepção "para a frente" e com isso outro trecho de código pode capturá-la e tratar essa excepção original retendo todas as informações necessárias com o stack trace.

- ***throw ex*** **- q**uando se utiliza o ***throw ex***, para-se a excepção ali e após se fazer alguma operação, é lançada outra excepção a partir desse ponto. Com isso, a informação de onde veio a excepção original é perdida, como se tivesse ocorrido uma nova excepção.

- De preferência devemos usar a primeira opção.

C# MANIPULAÇÃO DE EXCEPÇÕES - THROW



```
private int idade;
public int Idade {
    get { return idade; }
    set {
          if (value< 10 || value> 30)
             throw new Exception("Idade Inválida!");
          else
             idade = value;
    }
}
```

```
Aluno aluno = new Aluno(12342, "NomeAluno", Tipo.Normal);
Try {
      aluno.Idade = 50;
}
catch(Exception e) {
      Console.WriteLine(e.Message);
}
```

C# MANIPULAÇÃO DE EXCEÇPÕES – EX1

```
Aluno al = null;
    try {
        Console.WriteLine(al.ToString());
    }
    catch (ArgumentNullException ex) {
        Console.WriteLine("ArgumentNullException: " + ex.Message);
    }
    catch (ArgumentException ex) {
        Console.WriteLine("ArgumentException: " + ex.Message);
    }
    catch (Exception ex) {
        Console.WriteLine("Exception: " + ex.Message);
    }
    finally {
        Console.WriteLine("Executa sempre…");
    }
```

C# MANIPULAÇÃO DE EXCEPÇÕES – EX1



```
class DivNumeros
{
    int result;
    DivNumeros() {
        result = 0;
    }
    public void Divisao(int num1, int num2)
    {
        try {  result = num1 / num2;
        }
        catch (DivideByZeroException e)
            { Console.WriteLine("{0}", e);
        }
        finally {
            Console.WriteLine("Resultado: {0}", result);
        }
    }
}
```

O que aconteceria se se fizesse esta chamada?

```
static void Main(string[] args)
 {
  DivNumeros d = new DivNumeros();
  d.Divisao(25, 0);
  Console.ReadKey();
 }
```

Iria tentar executar uma divisão por Zero
O que é impossível e como tal geraria uma excepção

C# MANIPULAÇÃO DE EXCEPÇÕES – EX2

```
public class MaiorIdadeException : Exception
    {
        public MaiorIdadeException(string message) : base(message)
        { }
    }
```

```
public class Idade
    {
        int idade = 12;

        public void mostraIdade()
        {
            if (idade < 18)
            {
                throw (new MaiorIdadeException("Idade inferior a 18 anos!"));
            }
            else
            {
                Console.WriteLine("Idade: {0}", idade);
            }
        }
    }
```

C# MANIPULAÇÃO DE EXCEPÇÕES – EX3



```
class VerificaIdade
{
    static void Main(string[] args)
    {
        Idade idade = new Idade();
        try
        {
            idade.mostraIdade();
        }
        catch (MaiorIdadeException e)
        {
            Console.WriteLine("MaiorIdadeException: {0}", e.Message);
        }
        Console.ReadKey();
    }
}
```

```
public class MaiorIdadeException : Exception
{
        public MaiorIdadeException(string message) : base(message)
        { }
}
```